

OpenClaw Three Dialogue Configuration Tests

OpenClaw Three Dialogue Configuration Tests

1. Course Overview
2. Three Dialogue Schemes Introduction
 - 2.1 WhatsApp Scheme
 - 2.2 TUI Scheme
 - 2.3 AI Voice Interaction Scheme
3. WhatsApp Configuration and Testing
 - 3.1 Configuration Check
 - 3.2 Testing Method
 - 3.3 Common Problems
4. TUI Dialogue Configuration and Testing
 - 4.1 Startup Method
 - 4.2 Testing Method
 - 4.3 Local Hardware Command Verification
5. AI Voice Interaction Configuration and Testing
 - 5.1 Pre-Startup Preparation
 - 5.2 Startup Method
 - 5.3 Testing Method
 - 5.4 Common Problems
6. Comparison of Three Schemes
7. Recommended Test Sequence
8. Troubleshooting Principles

1. Course Overview

This course is for Jetson Nano Docker environment, introducing three common dialogue methods for OpenClaw, and providing their respective configuration checks and testing methods:

- WhatsApp dialogue
- Local TUI dialogue
- AI voice interaction dialogue

After completing this course, you can determine whether the TUI, WhatsApp, and voice三条 chains are normal, and distinguish between "model did not generate hardware actions" and "hardware execution failed" these two types of problems.

Note: TUI and WhatsApp use OpenClaw gateway's default model configuration; AI voice interaction also involves ASR/TTS/voice model configuration in the ROS2 voice package. When changing the default LLM, first confirm `/root/.openclaw/openclaw.json` and OpenClaw auth profile.

2. Three Dialogue Schemes Introduction

2.1 WhatsApp Scheme

Suitable scenarios:

- Mobile phone remote control
- Classroom demonstration or multi-person experience
- Remote viewing of OpenClaw replies

Features:

- No need to stay by the Jetson all the time
- Uses the same OpenClaw agent/model configuration as TUI

2.2 TUI Scheme

Suitable scenarios:

- Local debugging
- Quick testing of hardware actions
- Troubleshooting model, skill, atomic tool/SKILL chain, exec call issues

Features:

- Most direct
- Easiest to view commands and return results
- Recommended as the first verification entry among the three schemes

2.3 AI Voice Interaction Scheme

Suitable scenarios:

- Voice wake-up and voice control
- Desktop assistant-style experience
- Complete OpenClaw interaction demonstration

Features:

- Strongest sense of interaction
- Chain is longer than TUI/Whatsapp: Microphone/ASR -> ROS2 largemodel -> openclaw_bridge -> OpenClaw gateway -> TTS/speaker
- If "recognized but peripherals have no action", priority check whether the bridge layer generated OpenClaw hardware actions

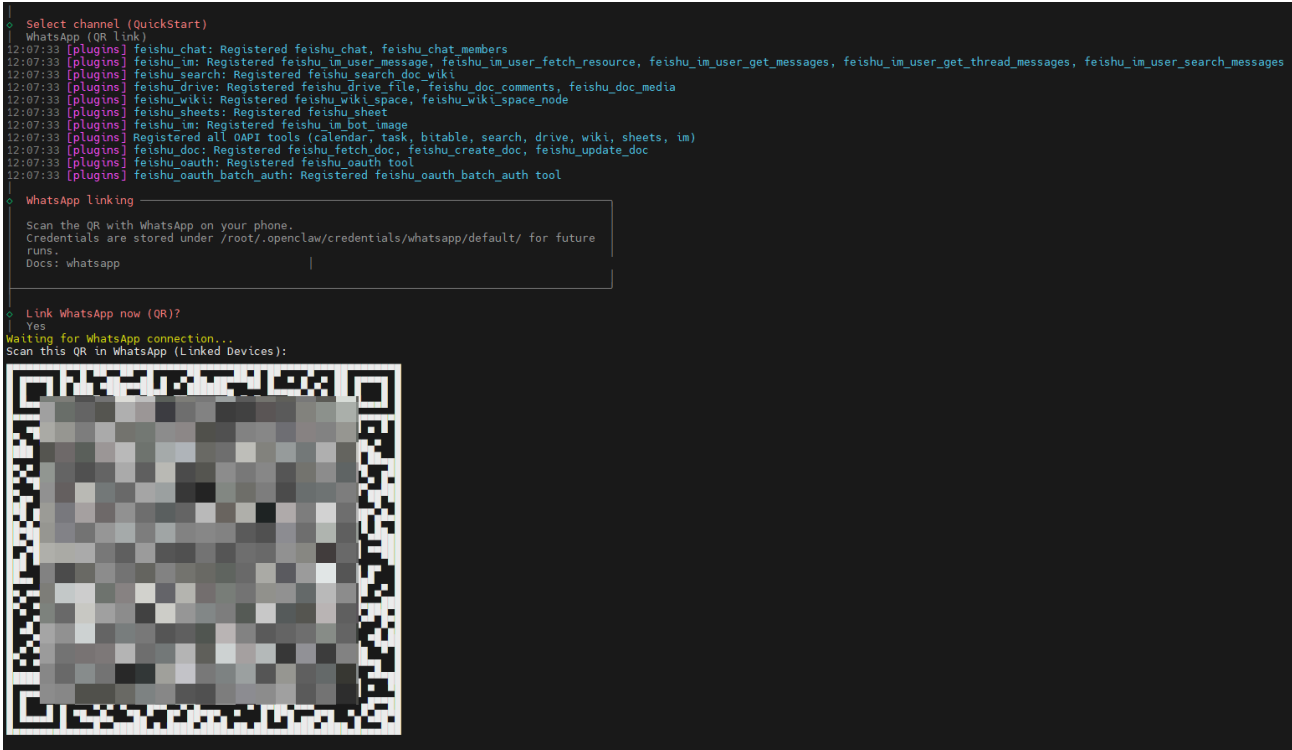
3. WhatsApp Configuration and Testing

3.1 Configuration Check

OpenClaw install WhatsApp plugin command:

```
openclaw channels login --channel whatsapp
```

After execution, a QR code usually appears as follows:



Scan the QR code with your phone to connect. When "Linked after restart" appears, it proves the connection is successful

```
WhatsApp asked for a restart after pairing (code 515); creds are saved. Restarting connection once...  
☑ Linked after restart; web session ready.
```

Open the newly created conversation to see the conversation ID



Fill this into the "allowFrom" under the openclaw.json file:

```
/root/.openclaw/openclaw.json
```

```
{} openclaw.json X
.openclaw > {} openclaw.json > {} channels > {} feishu > abc chat_id
144     "commands": {
145         "native": "auto",
146         "nativeSkills": "auto",
147         "restart": true,
148         "ownerDisplay": "raw"
149     },
150     "session": {
151         "dmScope": "per-channel-peer"
152     },
153     "channels": {
154         "whatsapp": {
155             "enabled": true,
156             "dmPolicy": "allowlist",
157             "selfChatMode": true,
158             "allowFrom": [
159                 "+8615338857526"
160             ],
161             "groupPolicy": "allowlist",
162             "debounceMs": 0,
163             "mediaMaxMb": 50
164         },
```

Execute the following command in terminal to restart openclaw gateway:

```
openclaw gateway restart
```

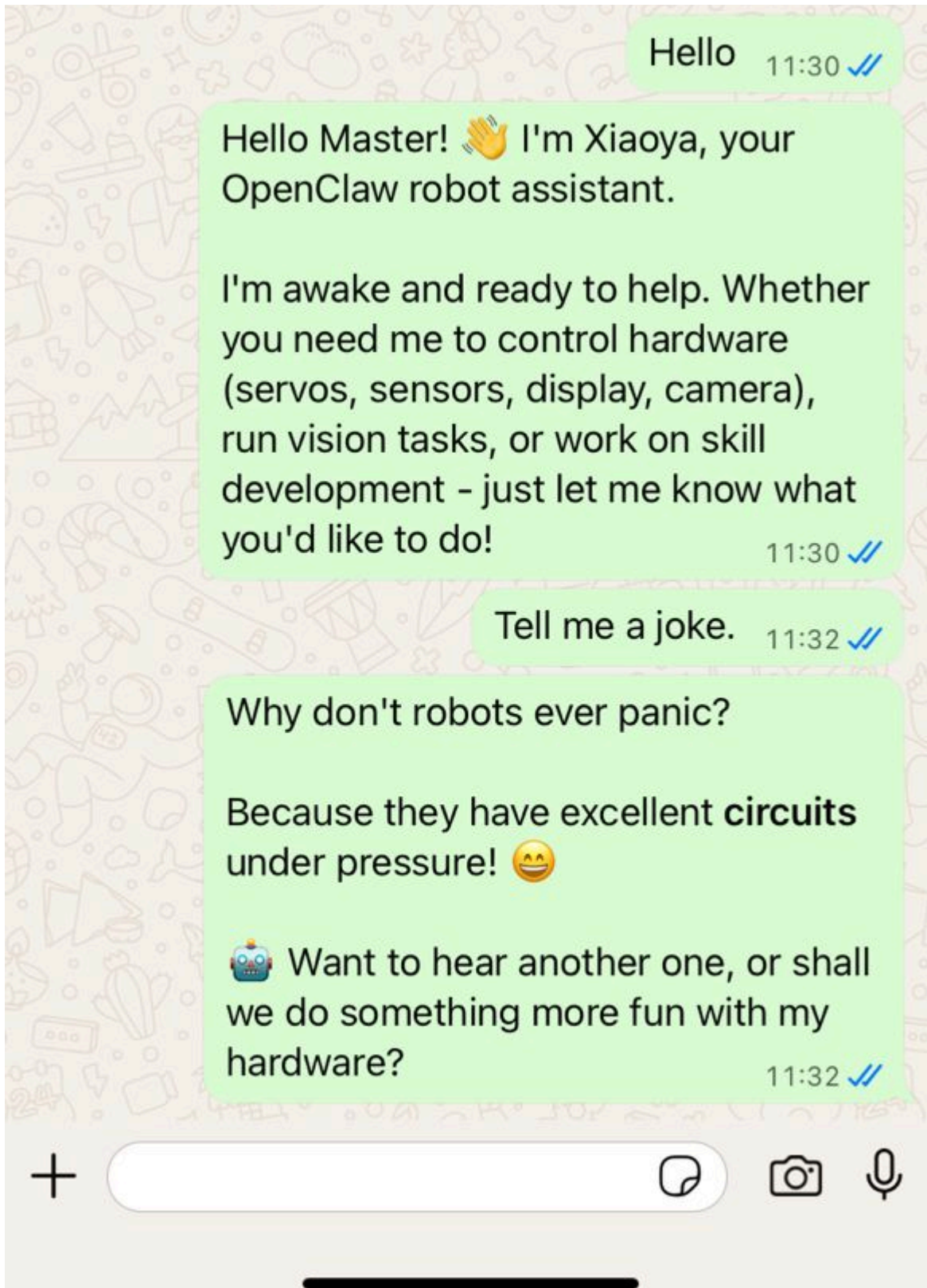
You can use the following command to view current openclaw related processes:

```
ps aux | grep openclaw
```

3.2 Testing Method

After creation is complete, you can directly send test messages in WhatsApp, for example:

- Tell me a joke.



3.3 Common Problems

- Cannot create bot after scanning: Check network first
- Creation successful but no reply: Check if OpenClaw gateway has restarted completely

4. TUI Dialogue Configuration and Testing

4.1 Startup Method

Enter Jetson Docker container:

```
~/run_usb_docker.sh
```

Start TUI:

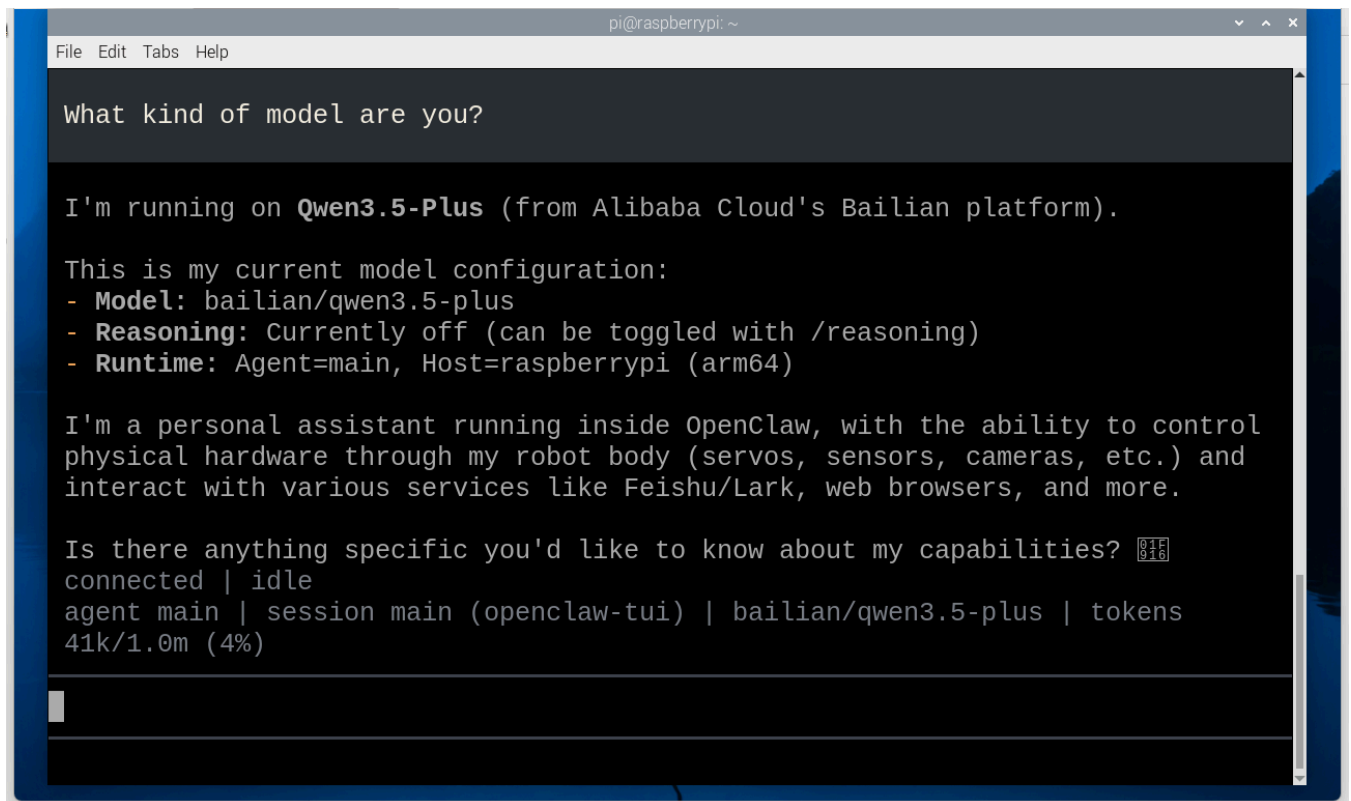
```
openclaw tui
```

If startup is successful, you will enter the local terminal dialogue interface.

4.2 Testing Method

It is recommended to first input normal dialogue:

- Tell me a joke
- Introduce yourself



Then test basic hardware:

- Turn RGB light red
- Display current temperature and humidity on OLED
- Take a photo

Then test combined actions:

- Turn two servos to 180 degrees, then RGB red breathing light, then take photo, finally display

temperature and humidity on OLED

Then test AI writing scripts to control peripherals:

- Help me write a script saved to workspace/scripts, to get temperature and humidity displayed on OLED, while RGB lights up green
- Help me write a script, two servos rotate to 180 degrees, then RGB lights up red breathing light, then takes a photo, finally gets temperature and humidity displayed on OLED, save to workspace/scripts

After script is generated successfully, the expected reply should contain the save path, for example:

```
/root/.openclaw/workspace/scripts/openclaw_rgb_dht_oled.py
```

Unless the user explicitly requests to run, script generation requests should only save the script and should not directly trigger hardware actions.

- Please write a script for me to put on my desktop, so that it can obtain temperature and humidity data, display it on an OLED screen, and simultaneously illuminate a green RGB light.

帮我写个脚本放在桌面，实现获取温湿度显示在OLED上，同时RGB亮绿灯

脚本已保存: /root/Desktop/openclaw_rgb_dht_oled.py。运行命令: python3 /root/Desktop/openclaw_rgb_dht_oled.py

全部舵机转到180度，然后 RGB 红色呼吸灯，然后拍照，最后把温湿度显示到 OLED

```
- 全部舵机 -> 180度: 完成
- RGB red breathe: 完成
- 读取温湿度并显示到 OLED: 已显示, 30.3°C, 49.0%
- 拍照: 已保存 /root/.openclaw/media/csi-camera/capture-20260529-165632.jpg
connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.5-plus | tokens 54k/32k (169%)
```

4.3 Local Hardware Command Verification

If you suspect the model is not correctly generating actions, you can bypass the model to test the wrapper directly:

```
/usr/local/bin/dht --format json    # Get sensor data
/usr/local/bin/dht_oled --format json # Sensor data displayed on OLED
/usr/local/bin/rgb --format json color --color red #All turn red
/usr/local/bin/rgb --format json effect --effect breathe --color blue --speed 5 #Blue
breathing light
/usr/local/bin/servo call-tool servo1 --action set_angle --angle 90 #Servo1 rotates to 90
degrees
/usr/local/bin/oled text --text 'Hello openClaw'
/usr/local/bin/oled clear
/usr/local/bin/csi_snap snap
/usr/local/bin/csi_snap snap --width 1280 --height 720
```

If these commands can execute but TUI dialogue has no action, the problem is usually in the model's thinking planning or skill selection, not the hardware connection.

5. AI Voice Interaction Configuration and Testing

5.1 Pre-Startup Preparation

Requirements:

- AI voice interaction module
- Speaker
- ROS2 `largemodel` package available in Jetson Docker container
- OpenClaw gateway configuration is online

Check key paths:

```
ls /root/yahboom_ws/src/largemodel
ls /root/yahboom_ws/src/largemodel/config
ls /root/yahboom_ws/src/largemodel/launch/openclaw_bridge.launch.py
```

5.2 Startup Method

Enter container:

```
~/run_usb_docker.sh
```

Then start gateway first:

```
openclaw gateway
```

Then enter ROS2 workspace and start voice main program:

```
cd /root/yahboom_ws/src/largemodel
python3 main.py
```

5.3 Testing Method

First test normal voice:

- Tell me a joke
- What model are you

Then test hardware voice:

- Turn on RGB red light
- Servo1 rotates to 90 degrees
- Read temperature and humidity and display on OLED screen

5.4 Common Problems

- No response after wake-up: Priority check voice module, serial port, or wake-up configuration
- Can wake up but cannot recognize: Check ASR model, API key, network
- Can recognize but not execute: Check if `openclaw_bridge` is passing ASR text to OpenClaw, check if OpenClaw generated hardware actions
- Has reply but no broadcast: Check `tts_text_input` topic, TTS configuration, and audio output device
- Voice recognition recognizes `RGB` as `rb`: Need to do common misrecognition normalization in bridge or router
- Temperature and humidity reading failure: First verify sensor with `dht`, then determine if it's a voice chain problem

6. Comparison of Three Schemes

Scheme	Suitable Scenario	Advantages	Recommended Position
TUI	Local debugging, command verification, troubleshooting model calls	Most direct, easiest to locate problems	Preferred basic debugging solution
WhatsApp	Remote control, classroom demonstration, multi-person experience	Convenient on mobile phone, suitable for demonstrating remote control	Demonstrate "remote natural language control"
AI Voice Interaction	Voice wake-up, complete desktop assistant experience	Strongest sense of interaction	Final demonstration solution

7. Recommended Test Sequence

It is recommended to complete the three dialogue scheme tests in the following order:

1. First test `openclaw_tui`
2. Then test WhatsApp
3. Finally test AI voice interaction

Reason:

- TUI has the shortest chain, suitable for first confirming model, skill, and atomic tools
- WhatsApp adds channel configuration on top of TUI, suitable for second step verification
- Voice chain is the longest, including ASR/TTS/ROS2 bridge, suitable for final verification

8. Troubleshooting Principles

When peripherals have no action, troubleshoot in the following order:

1. Can hardware commands be used normally
2. Check gateway printed messages to see if input is correctly recognized
3. Finally check hardware connection, power, I2C, camera, audio device